

UNIDAD 7

PROGRAMACIÓN ORIENTADA A OBJETOS (AVANZADA)

**Programación
CFGS DAW**

Adaptado y actualizado por: José Miguel Blázquez
Autores: Carlos Cacho, Raquel Torres y Lionel Tarazón

2023/2024

Licencia



Reconocimiento - NoComercial - CompartirIgual (by-nc-sa): No se permite un uso comercial de la obra original ni de las posibles obras derivadas, la distribución de las cuales se debe hacer con una licencia igual a la que regula la obra original.

Nomenclatura

A lo largo de este tema se utilizarán distintos símbolos para distinguir elementos importantes dentro del contenido. Estos símbolos son:



Importante



Atención



Interesante

ÍNDICE DE CONTENIDO

1. La clase ArrayList.....	4
1.1 Introducción.....	4
1.2 Declaración.....	4
1.3 Inserción de elementos.....	4
1.4 Recorrido de un ArrayList.....	4
1.5 Ordenar un ArrayList.....	5
1.6 Métodos de acceso y Manipulación.....	5
1.7 Ejemplo 1.....	7
1.8 Ejemplo 2.....	8
2. Asociación, agregación y composición: relaciones entre objetos.....	10
2.1 Asociación.....	10
2.2 Agregación.....	11
2.3 Composición.....	12
3. Herencia: relaciones entre clases.....	14
3.1 Introducción.....	14
3.2 Constructores de clases derivadas.....	15
3.3 Métodos heredados y sobrescritos.....	15
3.4 Clases y métodos final.....	15
3.5 Acceso a miembros derivados.....	15
3.6 Ejemplo 3.....	16
4. Polimorfismo.....	20
4.1 Ejemplo 4.....	20
4.2 Tipos de polimorfismo.....	21
5. Clases abstractas.....	24
6. Interfaces.....	25
6.1 Ejemplo básico de interfaces.....	26
6.2 Ejemplo: interfaz estándar para ordenar objetos complejos.....	28

UD07. PROGRAMACIÓN ORIENTADA A OBJETOS (AVANZADA)

1. LA CLASE ARRAYLIST

La Clase ArrayList no está directamente relacionada con la programación orientada a objetos, pero este es un buen momento para aprender a utilizarla debido a su potencial.

1.1 Introducción

Un *ArrayList* es objeto que representa una estructura de datos dinámica del tipo **colección** que implementa una lista de tamaño variable. Es similar a un *array estático* con las ventajas de que **su tamaño puede crecer dinámicamente conforme se añaden elementos** (no es necesario fijar su tamaño al crearlo) y **permite almacenar datos y objetos de cualquier tipo** (aunque esto último hay que evitarlo).

1.2 Declaración

Un *ArrayList* se declara como una clase más: `ArrayList lista = new ArrayList();`

Sin embargo, es preferible usar esta declaración: `ArrayList<E> lista = new ArrayList<>();`

Donde E denota el tipo de datos que vamos a almacenar. Ejemplos:

```
ArrayList<Integer> numeros = new ArrayList<>();  
ArrayList<Persona> empleados = new ArrayList<>();
```

* Necesitaremos importarla para poder usarla: `import java.util.ArrayList;`

1.3 Inserción de elementos

Para insertar datos o elementos en un ArrayList puede utilizarse el método **add()**. En el siguiente ejemplo insertamos datos sobre los 2 ArrayList declarados:

```
numeros.add(1);  
numeros.add(500);  
empleados.add(persona1); //Debe existir persona1  
empleados.add(new Persona("Paco", 25));
```

En el caso de “empleados”, cuando añadimos un elemento al ArrayList realmente estamos insertando “punteros” o direcciones de memoria de los propios objetos creados. En el ejemplo anterior, se muestran 2 formas de añadir personas al ArrayList.

1.4 Recorrido de un ArrayList

Hay varias maneras diferentes para iterar (recorrer) una colección del tipo ArrayList:

- Utilizando el bucle *for* y el método *get()* con un índice.

```
for(int i = 0; i < lista.size(); i++) {  
    System.out.println(lista.get(i)); // Lo imprimimos por pantalla  
}
```

- Utilizando el bucle *foreach*.

```
for(String palabra: lista)  
    System.out.println(palabra);
```

- Usando un objeto *Iterator* que permite recorrer listas como si fuese un índice (más adelante exploraremos este concepto). Se necesita importar la clase con *import java.util.Iterator;* Tiene 2 métodos principales:
 - hasNext()*: devuelve true si hay más elementos en el array.
 - next()*: devuelve el siguiente objeto contenido en el array.

```
ArrayList<Integer> numeros = new ArrayList<>();  
//una vez insertados elementos. . .  
Iterator <Integer> iterator = numeros.iterator(); //crear iterador  
while(iterator.hasNext()) //mientras haya elementos  
    if (condición)  
        iterator.remove(); //se obtienen y se borran
```

Lo habitual será usar *foreach*, para recorridos completos, *for* (*int i*... para recorridos específicos (solo algunas posiciones, orden descendente, etc.) y los iteradores para realizar borrados o modificaciones sustanciales de la estructura, ya que no se deben usar los 2 primeros métodos.

1.5 Ordenar un ArrayList

Los ArrayList de objetos simples como puedan ser String, Integer, Float, LocalDate, etc. se pueden ordenar de forma sencilla mediante la clase *Collection* (debemos hacer un: *import java.util.Collections;*) :

```
ArrayList <Miclase> lista = new ArrayList<>();  
...  
Collections.sort(lista);
```

O bien, en orden inverso:

```
Collections.sort(lista, Collections.reverseOrder());
```

Ordenar un ArrayList de objetos creados por nosotros es más complicado y lo veremos al final de esta unidad.

1.6 Métodos de acceso y Manipulación

Un *ArrayList* da a cada elemento insertado un índice numérico que hace referencia a la posición donde se encuentra, de forma similar a un array. Así, el primer elemento se encuentra almacenado en el índice 0, el segundo en el 1, el tercero en el 2 y así sucesivamente.

Los métodos más utilizados para acceder y manipular un *ArrayList* son:

<code>int size()</code>	Devuelve el número de elementos de la lista.
<code>E get(int index)</code>	Devuelve una referencia al elemento en la posición <code>index</code>
<code>void clear()</code>	Elimina todos los elementos de la lista. Establece el tamaño a cero
<code>boolean isEmpty()</code>	Devuelve <code>true</code> si la lista no contiene elementos
<code>boolean add(E element)</code>	Inserta <code>element</code> al final de la lista y devuelve <code>true</code>
<code>void add(int index, E element)</code>	Inserta <code>element</code> en la posición <code>index</code> de la lista. Desplaza una posición todos los demás elementos de la lista (no sustituye ni borra otros elementos)
<code>void set(int index, E element)</code>	Sustituye el elemento en la posición <code>index</code> por <code>element</code>
<code>boolean contains(Object o)</code>	Busca el objeto <code>o</code> en la lista y devuelve <code>true</code> si existe. Utiliza el método <code>equals()</code> para comparar objetos (<i>si no está sobrescrito no funcionará</i>).
<code>int indexOf(Object o)</code>	Busca el objeto <code>o</code> en la lista, empezando por el principio, y devuelve el índice dónde se encuentre. Devuelve <code>-1</code> si no existe. Utiliza <code>equals()</code> para comparar objetos (<i>si no está sobrescrito no funcionará</i>).
<code>int lastIndexOf(Object o)</code>	Como <code>indexOf()</code> pero busca desde el final de la lista.
<code>E remove(int index)</code>	Elimina el elemento en la posición <code>index</code> y lo devuelve (<i>si no está sobrescrito no funcionará</i>).
<code>boolean remove(Object obj)</code>	Elimina la primera ocurrencia de <code>obj</code> en la lista. Devuelve <code>true</code> si lo ha encontrado y eliminado, <code>false</code> en otro caso. Utiliza <code>equals()</code> para comparar objetos
<code>void remove(int index)</code>	Elimina el objeto de la lista que se encuentra en la posición <code>index</code> . Es más rápido que el método anterior ya que no necesita recorrer toda la lista.

Consulta todo en: [Documentación oficial de ArrayList \(Java 11 API\)](#)

BONUS: también existen los **ArrayList N-dimensionales**. Veamos un ejemplo bidimensional (como si fuera una tabla o matriz).

```
ArrayList<ArrayList<Integer>> tablaNumeros = new ArrayList<>();
tablaNumeros.add(new ArrayList<>());
tablaNumeros.get(fila).add(valor);
tablaNumeros.get(fila).get(columna);
```

1.7 Ejemplo 1

Ejemplo que crea, rellena y recorre un ArrayList de dos formas diferentes.

```
import java.util.ArrayList;

public class ArrayListEjemplo1 {

    public static void main(String[] args) {
        // Crear un ArrayList de Strings
        ArrayList<String> listaStrings = new ArrayList<>();

        // Forma 1: Añadir elementos uno por uno usando el método add()
        listaStrings.add("Elemento 1");
        listaStrings.add("Elemento 2");
        listaStrings.add("Elemento 3");

        // Forma 2: Añadir elementos usando addAll() con otra colección
        ArrayList<String> otraLista = new ArrayList<>();
        otraLista.add("Elemento 4");
        otraLista.add("Elemento 5");
        listaStrings.addAll(otraLista);

        // Recorrer y mostrar los elementos usando un bucle foreach
        for (String elemento : listaStrings) {
            System.out.println(elemento);
        }

        // Recorrer y mostrar los elementos usando un bucle for tradicional
        for (int i = 0; i < listaStrings.size(); i++) {
            System.out.println(listaStrings.get(i));
        }
    }
}
```

1.8 Ejemplo 2

Tenemos la clase Producto con:

- 2 atributos: nombre (String) y cantidad (int).
- Un constructor con parámetros.
- Un constructor sin parámetros.
- Métodos get y set asociados a los atributos.

```
class Producto {  
    private String nombre;  
    private int cantidad;  
  
    // Constructor con parámetros  
    public Producto(String nombre, int cantidad) {  
        this.nombre = nombre;  
        this.cantidad = cantidad;  
    }  
  
    // Constructor sin parámetros  
    public Producto() {  
        this.nombre = "";  
        this.cantidad = 0;  
    }  
  
    // Métodos get y set  
    public String getNombre() {  
        return nombre;  
    }  
  
    public void setNombre(String nombre) {  
        this.nombre = nombre;  
    }  
  
    public int getCantidad() {  
        return cantidad;  
    }  
  
    public void setCantidad(int cantidad) {  
        this.cantidad = cantidad;  
    }  
}
```


En el programa principal creamos una lista de productos y realizamos operaciones sobre ella:

```
public class ArrayListEjemplo2 {
    public static void main(String[] args) {
        // Crear un ArrayList de Productos
        ArrayList<Producto> listaProductos = new ArrayList<>();

        // Añadir productos al ArrayList
        listaProductos.add(new Producto("Producto1", 10));
        listaProductos.add(new Producto("Producto2", 5));
        listaProductos.add(new Producto("Producto3", 8));
        listaProductos.add(new Producto("Producto4", 15));
        listaProductos.add(new Producto("Producto5", 20));

        System.out.println("Lista original:");
        for (Producto producto : listaProductos) {
            System.out.println("Nombre: " + producto.getNombre() + ", Cantidad: " + producto.getCantidad());
        }

        Producto nuevoProducto = new Producto("Producto6", 12);
        listaProductos.add(nuevoProducto); // Añadir un nuevo producto

        System.out.println("\nLista después de añadir un producto:");
        for (Producto producto : listaProductos) {
            System.out.println("Nombre: " + producto.getNombre() + ", Cantidad: " + producto.getCantidad());
        }

        // Buscar un producto por nombre y eliminarlo
        String nombreProdEliminar = "Producto3";
        for (int i = 0; i < listaProductos.size(); i++) {
            if (listaProductos.get(i).getNombre().equals(nombreProdEliminar)) {
                listaProductos.remove(i);
                break;
            }
        }

        // Intenta hacer un bucle donde, a mitad de iteración, añadas un
        // elemento...¿qué sucede?, ¿porqué?, ¿cómo lo resolverías?
    }
}
```

2. ASOCIACIÓN, AGREGACIÓN Y COMPOSICIÓN: RELACIONES ENTRE OBJETOS

Cuando avanzamos en la complejidad de las aplicaciones, dentro del paradigma orientado a objetos, aparecen varios objetos que interactúan y colaboran entre ellos para conseguir la resolución de un problema más o menos complejo. Por ello, es fundamental estudiar los mecanismos existentes para establecer las relaciones entre objetos. Existen 3 términos fundamentales, que son los que vamos a describir: asociación, agregación y composición.

Las asociaciones representan conexiones entre objetos (instancias), reflejando interacciones y dependencias en el sistema. Por otro lado, las agregaciones y composiciones van más allá al definir la naturaleza y la duración de las relaciones entre objetos, permitiéndonos diseñar sistemas con mayor precisión y control.

2.1 Asociación

En una asociación, dos instancias relacionadas entre sí existen de forma independiente, donde una clase simplemente conoce a la otra. También considerada como una “relación débil”. La creación o desaparición de uno de ellos implica únicamente la creación o destrucción de la relación entre ellos y nunca la creación o destrucción del otro. Por ejemplo, una persona tiene asociada una dirección donde reside. Sin embargo, la existencia de ambas clases no se ve afectada mutuamente.

La asociación expresa una relación (unidireccional o bidireccional) entre las instancias a partir de las clases conectadas. El sentido en que se recorre la asociación se denomina navegabilidad de la asociación. Cada extremo de la asociación se caracteriza por el rol o papel que juega en dicha relación el objeto situado en cada extremo. La cardinalidad o multiplicidad es el número mínimo y máximo de instancias que pueden relacionarse con la otra instancia del extremo opuesto de la relación. Por defecto es 1. El formato en el que se especifica es (mínima..máxima).

Veamos un ejemplo sencillo mediante la relación entre una clase “Persona” y otra clase “Dirección”:

```
// Clase Direccion para la asociación
class Direccion {
    String calle;
    String ciudad;
    String estado;

    public Direccion(String calle, String ciudad, String estado) {
        this.calle = calle;
        this.ciudad = ciudad;
        this.estado = estado;
    }
}

// Clase Persona con una asociación a la clase Direccion
class Persona {
```

```
String nombre;  
Direccion direccion;  
  
public Persona(String nombre, Direccion direccion) {  
    this.nombre = nombre;  
    this.direccion = direccion;  
}  
}
```

2.2 Agregación

En una relación todo-parte una instancia forma parte de otra. En la vida real se dice que A está compuesto de B o que A tiene B. La diferencia entre asociación y relación todo-parte radica en la asimetría presente en toda relación todo-parte. En teoría se distingue entre 2 tipos de relación todo-parte: las agregaciones y las composiciones.

La agregación es una relación más fuerte que la asociación, es una relación binaria que representa una relación todo-parte (pertenecer a, tener, ser parte de). A nivel práctico se suele llamar agregación cuando la relación se plasma mediante referencias (lo que permite que un componente esté referenciado en más de un compuesto). Así, a nivel de implementación una agregación no se diferencia de una asociación binaria. Por ejemplo: una universidad y sus estudiantes.

Características de implementación:

- Para que sea agregación, la vida del objeto agregado no debe depender necesariamente del objeto agregador; es decir, que al destruirse el objeto agregador, eso no signifique que se tenga que destruir también al objeto agregado.
- Eso implica que puede haber en el programa varias referencias al objeto agregado, no sólo la que almacena el agregador.
- Lo habitual en la agregación es que la variable de instancia que almacene la referencia al objeto agregado se asigne, o bien directamente (si la variable tiene la suficiente visibilidad) o bien a través de un método que reciba la referencia y se la asigne a la variable de instancia.
- Ese método puede ser (y suele ser) un constructor de la clase.

En este caso, una clase contiene a la otra, pero ambas pueden existir de manera independiente. Vamos a representar esto mediante una relación entre una clase Universidad y una clase Estudiante.

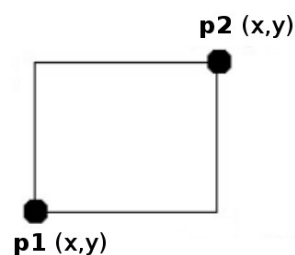
```
// Clase Estudiante para la agregación  
class Estudiante {  
    String nombre;  
    int edad;  
  
    public Estudiante(String nombre, int edad) {  
        this.nombre = nombre;  
        this.edad = edad;  
    }  
}
```

```
    }  
}  
  
// Clase Universidad con una agregación a la clase Estudiante  
class Universidad {  
    String nombre;  
    List<Estudiante> estudiantes;  
  
    public Universidad(String nombre) {  
        this.nombre = nombre;  
        this.estudiantes = new ArrayList<>();  
    }  
  
    public void agregarEstudiante(Estudiante estudiante) {  
        estudiantes.add(estudiante);  
    }  
}
```

2.3 Composición

La composición es una **agregación fuerte** en la que una instancia 'parte' está relacionada, como máximo, con una instancia 'todo' en un momento dado, de forma que cuando un objeto 'todo' es eliminado, también son eliminados sus objetos 'parte'. Es decir, cuando la relación se conforma en una inclusión por valor (lo que implica que un componente está como mucho en un compuesto, pero no impide que haya objetos componentes no relacionados con ningún compuesto). En este caso si se destruye el compuesto se destruyen sus componentes. Por ejemplo: un rectángulo está compuesto por 2 puntos, pero si se "destruye" alguno afectaría al rectángulo...

```
public class Rectangulo {  
    Punto p1;  
    Punto p2;  
    ...  
}  
  
public class Punto {  
    int x, y;  
    ...  
}
```



La composición de clases es una capacidad muy potente de la POO ya que permite diseñar software como un conjunto de clases que colaboran entre sí: Cada clase se especializa en una tarea concreta y esto permite dividir un problema complejo en varios sub-problemas pequeños. También facilita la modularidad y reutilización del código.

Características de implementación:

- Para que sea composición, la vida del objeto componente debe depender necesariamente del objeto compuesto; es decir, que al destruirse el objeto compuesto, se debe destruir

también al objeto componente.

- Eso implica que sólo puede haber en el programa una sola referencia al objeto componente, que es la que almacena el objeto compuesto.
- Por eso, lo habitual en la composición es que el objeto compuesto sea el responsable de crear al objeto componente.
- Normalmente, no hay setters para el componente y, en caso de haber getters, deberían devolver una copia del objeto componente, y no el objeto componente original.

En resumen, la composición es la relación más fuerte, donde una clase está compuesta por la otra y su existencia está vinculada. Vamos a representar esto mediante una relación entre una clase Motor y una clase Automóvil.

```
// Clase Motor para la composición
class Motor {
    String tipo;

    public Motor(String tipo) {
        this.tipo = tipo;
    }
}

// Clase Automovil con una composición de la clase Motor
class Automovil {
    String modelo;
    Motor motor;

    public Automovil(String modelo, String tipoMotor) {
        this.modelo = modelo;
        this.motor = new Motor(tipoMotor);
    }
}
```

3. HERENCIA: RELACIONES ENTRE CLASES

3.1 Introducción

La herencia es una de las capacidades más importantes y distintivas de la POO. Consiste en obtener (**extender**) **una clase nueva a partir de otra ya existente de forma que la nueva clase “hereda” todos los atributos y métodos de la clase ya existente**. A la clase ya existente se la denomina **superclase**, clase **base** o clase **padre**. A la nueva clase se la denomina **subclase**, clase **derivada** o clase **hija**. En Java, **SÓLO se puede heredar de 1 clase**, no existe la herencia múltiple.

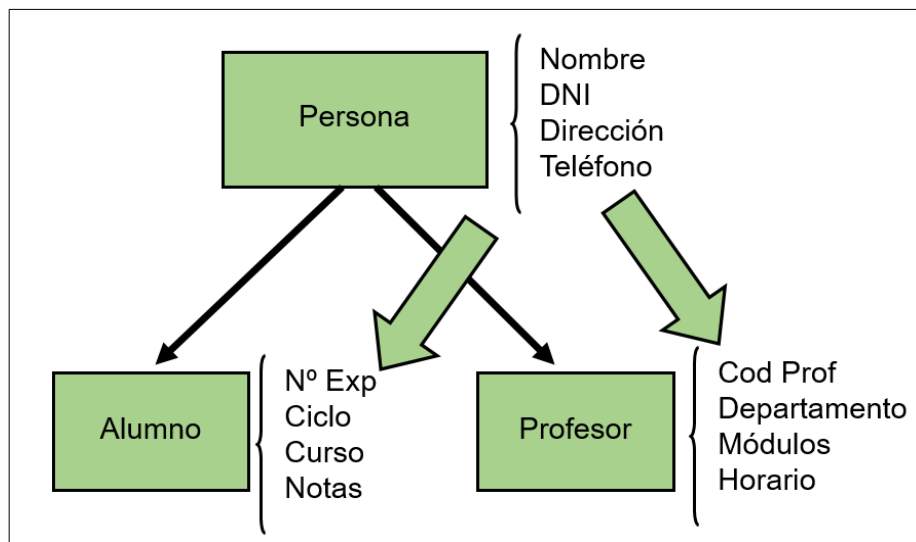


Cuando derivamos (o extendemos) una nueva clase, ésta hereda todos los datos y métodos miembro de la clase existente.

Por ejemplo: si tenemos un programa que va a trabajar con alumnos y profesores, éstos van a tener atributos comunes como el nombre, dni, dirección o teléfono. Pero cada uno de ellos tendrán atributos específicos que no tengan los otros. Por ejemplo los alumnos tendrán el número de expediente, el ciclo y curso que cursan y sus notas; por su parte los profesores tendrán el código de profesor, el departamento al que pertenecen, los módulos que imparten y su horario.

Por lo tanto, en este caso lo mejor es declarar una clase *Persona* con los atributos comunes (Nombre, DNI, Dirección, Teléfono) y dos sub-clases *Alumno* y *Profesor* que hereden de *Persona* (además de tener sus propios atributos).

Es importante recalcar que *Alumno* y *Profesor* también heredarán todos los métodos de *Persona*.



En Java se utiliza la palabra reservada **extends** para indicar herencia:

```
public class Alumno extends Persona {}
```

```
public class Profesor extends Persona {}
```

3.2 Constructores de clases derivadas

El constructor de una clase derivada debe encargarse de construir los atributos que estén definidos en la clase base además de sus propios atributos. Dentro del constructor de la clase derivada, para llamar al constructor de la clase base se debe utilizar el método reservado **super()** pasándole como argumento los parámetros que necesite. Si no se llama explícitamente al constructor de la clase base mediante **super()** el compilador llamará automáticamente al constructor por defecto de la clase base. Si no tiene constructor por defecto el compilador generará un error.

```
public Alumno(String nom, String dni, String direcc, String telef, int numExp) {  
    super(nom, dni, direcc, telef); // Llamada al constructor de Persona  
    this.numExp = numExp;  
}
```

3.3 Métodos heredados y sobreescritos

Hemos visto que una subclase hereda los atributos y métodos de la superclase; además, se pueden incluir nuevos atributos y nuevos métodos. Por otro lado, puede ocurrir que alguno de los métodos que existen en la superclase no nos sirvan en la subclase (tal y como están programados) y necesitemos adecuarlos a las características de la subclase. Esto puede hacerse mediante la sobreescritura de métodos (polimorfismo):

Un método está sobreescrito o reimplementado cuando se programa de nuevo en la clase derivada. Por ejemplo el método *mostrarPersona()* de la clase *Persona* lo necesitaríamos sobreescibir en las clases *Alumno* y *Profesor* para mostrar también los nuevos atributos. El método sobreescrito en la clase derivada podría reutilizar el método de la clase base, si es necesario, y a continuación imprimir los nuevos atributos. En Java podemos acceder a métodos definidos en la clase base mediante `super.metodo()`.

3.4 Clases y métodos final

-  Una clase **final** no puede ser heredada por otras.
-  Un método **final** no puede ser sobreescrito por las subclases.

3.5 Acceso a miembros derivados

Aunque una subclase incluye todos los miembros de su superclase, no podrá acceder a aquellos que hayan sido declarados como *private*. Si en el ejemplo 3 intentásemos acceder desde las clases derivadas a los atributos de la clase *Persona* (que son privados) obtendríamos un error de compilación. También podemos declarar los atributos como *protected*. De esta forma podrán ser accedidos desde las clases heredadas, (nunca desde otras clases).

-  Los atributos declarados como **protected** son públicos para las clases heredadas y privados para las demás clases.

3.6 Ejemplo 3

En este ejemplo vamos a crear la clase *Persona* y sus clases heredadas: *Alumno* y *Profesor*. En la **clase *Persona*** crearemos el constructor, un método para mostrar los atributos y los getters y setters. Las **clases *Alumno* y *Profesor*** heredarán de la clase *Persona* (utilizando la palabra reservada *extends*) y cada una tendrá sus propios atributos, un constructor que llamará también al constructor de la clase *Persona* (utilizando el método *super()*), un método para mostrar sus atributos, que también llamará al método de *Persona* y los getters y setters. Es interesante ver cómo se ha sobrescrito el método *mostrarPersona()* en las clases heredadas. El método se llama igual y hace uso de la palabra reservada *super* para llamar al método de *mostrarPersona()* de *Persona*. En la llamada del *main* tanto el objeto *a* (*Alumno*) como el objeto *profe* (*Profesor*) pueden hacer uso del método *mostrarPersona()*.

Persona

```
10 public class Persona {
11     private String nombre;
12     private String dni;
13     private String direccion;
14     private int telefono;
15
16     public Persona(String nom, String dni, String direc, int tel)
17     {
18         this.nombre = nom;
19         this.dni = dni;
20         this.direccion = direc;
21         this.telefono = tel;
22     }
23
24     public void mostrarPersona()
25     {
26         System.out.println("Nombre: " + this.nombre);
27         System.out.println("DNI: " + this.dni);
28         System.out.println("Dirección: " + this.direccion);
29         System.out.println("Teléfono: " + this.telefono);
30     }
31
32     public String getNombre() {
33         return nombre;
34     }
35
36     public void setNombre(String nombre) {
37         this.nombre = nombre;
38     }
39
40     public String getDni() {
41         return dni;
42     }
43
44     public void setDni(String dni) {
45         this.dni = dni;
46     }
47
48     public String getDireccion() {
49         return direccion;
50     }
51
52     public void setDireccion(String direccion) {
53         this.direccion = direccion;
54     }
55
56     public int getTelefono() {
57         return telefono;
58     }
59
60     public void setTelefono(int telefono) {
61         this.telefono = telefono;
62     }
63
64 }
```


Alumno

(hereda de Persona)

```
8  import java.util.ArrayList;
9  import java.util.Iterator;
10
11
12  public class Alumno extends Persona{
13
14      private int exp;
15      private String ciclo;
16      private int curso;
17      private ArrayList notas;
18
19      // Al constructor hemos de pasarle los atributos de la clase Alumno y la de Persona
20      public Alumno(String nom, String dni, String direc, int tel, int exp, String ciclo, int curso, ArrayList notas)
21      {
22          // Llamamos al constructor de la clase Persona
23          super(nom, dni, direc, tel);
24
25          this.exp = exp;
26          this.ciclo = ciclo;
27          this.curso = curso;
28          this.notas = notas;
29      }
30
31      public void mostrarPersona()
32      {
33          // Llamamos al método de la clase madre para que muestre los datos de Persona
34          super.mostrarPersona();
35
36          System.out.println("Núm. expediente: " + this.exp);
37          System.out.println("Ciclo: " + this.ciclo);
38          System.out.println("Curso: " + this.curso);
39          System.out.println("Notas:");
40          for( Iterator it = this.notas.iterator(); it.hasNext(); )
41          {
42              System.out.println("\tNota: " + it.next());
43          }
44      }
45      public int getExp() {
46          return exp;
47      }
48
49      public void setExp(int exp) {
50          this.exp = exp;
51      }
52
53      public String getCiclo() {
54          return ciclo;
55      }
56
57      public void setCiclo(String ciclo) {
58          this.ciclo = ciclo;
59      }
60
61      public int getCurso() {
62          return curso;
63      }
64
65      public void setCurso(int curso) {
66          this.curso = curso;
67      }
68
69      public ArrayList getNotas() {
70          return notas;
71      }
72
73      public void setNotas(ArrayList notas) {
74          this.notas = notas;
75      }
76  }
```

Profesor (hereda de Persona)

```
8 import java.util.ArrayList;
9 import java.util.Iterator;
10
11 public class Profesor extends Persona{
12
13     private int cod;
14     private String depto;
15     private ArrayList modulos;
16     private String horario;
17
18     // Al constructor hemos de pasarle los atributos de la clase Profesor y la de Persona
19     public Profesor(String nom, String dni, String direc, int tel, int cod, String depto, ArrayList mod, String horario)
20     {
21         // Llamamos al constructor de la clase Persona
22         super(nom, dni, direc, tel);
23
24         this.cod = cod;
25         this.depto = depto;
26         this.modulos = mod;
27         this.horario = horario;
28     }
29
30     public void mostrarPersona()
31     {
32         // Llamamos al método de la clase madre para que muestre los datos de Persona
33         super.mostrarPersona();
34
35         System.out.println("Código: " + this.cod);
36         System.out.println("Departamento: " + this.depto);
37         System.out.println("Horario: " + this.horario);
38         System.out.println("Modulos:");
39         for( Iterator it = this.modulos.iterator(); it.hasNext(); )
40         {
41             System.out.println("\tMódulo: " + it.next());
42         }
43     }
44
45     public int getCod() {
46         return cod;
47     }
48
49     public void setCod(int cod) {
50         this.cod = cod;
51     }
52
53     public String getDepto() {
54         return depto;
55     }
56
57     public void setDepto(String depto) {
58         this.depto = depto;
59     }
60
61     public ArrayList getModulos() {
62         return modulos;
63     }
64
65     public void setModulos(ArrayList modulos) {
66         this.modulos = modulos;
67     }
68
69     public String getHorario() {
70         return horario;
71     }
72
73     public void setHorario(String horario) {
74         this.horario = horario;
75     }
76 }
```

Programa Principal

```

8  import java.util.ArrayList;
9
10 public class Herencia {
11
12     public static void main(String[] args) {
13
14         // Probamos la clase persona
15         // Llamamos al constructor con el nombre, dni, dirección y teléfono
16         Persona p = new Persona("Pepe", "00000000T", "C/ Colón", 666666666);
17
18         System.out.println("Mostramos una persona");
19         p.mostrarPersona();
20
21         //Probamos la clase Alumno
22
23         // Creamos las notas
24         ArrayList notas = new ArrayList();
25
26         notas.add(7);
27         notas.add(9);
28         notas.add(6);
29
30         // Llamamos al constructor con el nombre, dni, dirección, teléfono, expediente, ciclo, curso y las notas
31         Alumno a = new Alumno("Maria", "123456782", "P/ Libertad", 11111111, 1, "DAW", 1, notas);
32
33         System.out.println("-----");
34         System.out.println("Mostramos un alumno");
35         a.mostrarPersona();
36
37
38         //Probamos la clase Profesor
39
40         // Creamos los módulos
41         ArrayList modulos = new ArrayList();
42
43         modulos.add("Programación");
44         modulos.add("Lenguajes de marcas");
45         modulos.add("Entornos de desarrollo");
46
47         // Llamamos al constructor con el nombre, dni, dirección, teléfono, expediente, ciclo, curso y las notas
48         Profesor profe = new Profesor("Juan", "00000001R", "C/ Java", 22222222, 3, "Informática", modulos, "Mañanas");
49
50         System.out.println("-----");
51         System.out.println("Mostramos un profesor");
52
53         profe.mostrarPersona();
54     }
55 }

```

Salida:

```

Output - Herencia (run) X
run:
Mostramos una persona
Nombre: Pepe
DNI: 00000000T
Dirección: C/ Colón
Teléfono: 666666666
-----
Mostramos un alumno
Nombre: Maria
DNI: 123456782
Dirección: P/ Libertad
Teléfono: 11111111
Núm. expediente: 1
Ciclo: DAW
Curso: 1
Notas:
    Nota: 7
    Nota: 9
    Nota: 6
-----

```

```

-----
Mostramos un profesor
Nombre: Juan
DNI: 00000001R
Dirección: C/ Java
Teléfono: 22222222
Código: 3
Departamento: Informática
Horario: Mañanas
Modulos:
    Módulo: Programación
    Módulo: Lenguajes de marcas
    Módulo: Entornos de desarrollo
BUILD SUCCESSFUL (total time: 0 seconds)

```

4. POLIMORFISMO

La sobreescritura de métodos constituye la base de uno de los conceptos más potentes de Java: la **selección dinámica de métodos**, que es un mecanismo mediante el cual la llamada a un método sobreescrito se resuelve en tiempo de ejecución y no durante la compilación. La selección dinámica de métodos es importante porque permite implementar el polimorfismo durante el tiempo de ejecución. Una variable de referencia a una superclase se puede referir a un objeto de una subclase. Java se basa en esto para resolver llamadas a métodos sobreescritos en el tiempo de ejecución.

Lo que determina la versión del método que será ejecutado es el tipo de objeto al que se hace referencia y no el tipo de variable de referencia.

El polimorfismo es fundamental en la programación orientada a objetos porque permite que una clase general especifique métodos que serán comunes a todas las clases que se deriven de esa misma clase. De esta manera las subclases podrán definir la implementación de alguno o de todos esos métodos. Si bien el polimorfismo es un concepto ligado a la herencia, existe un tipo, la sobrecarga, que no requiere la existencia de herencia.

La superclase proporciona todos los elementos que una subclase puede usar directamente. También define aquellos métodos que las subclases que se deriven de ella deben implementar por sí mismas. De esta manera, combinando la herencia y la sobreescritura de métodos, una superclase puede definir la forma general de los métodos que se usarán en todas sus subclases

4.1 Ejemplo 4

Probemos un ejemplo sencillo pero que resume todo lo importante del polimorfismo. Vamos a crear la **clase Madre** con un método *llamame()*. A continuación crearemos **dos clases derivadas** de ésta: **Hija1** e **Hija2**, sobreescribiendo el método *llamame()*. En el *main* crearemos un objeto de cada clase y los asignaremos a una variable de tipo *Madre* (llamada *madre2*) con la que llamaremos al método *llamame()* de los tres objetos.

Es importante observar que la variable Madre madre2 se puede asignar a objetos de clase Hija1 e Hija2. Esto es posible porque Hija1 e Hija2 también son de tipo Madre (debido a la herencia). También es importante ver que la variable Madre madre2 llamará al método llamame() de la clase del objeto al que hace referencia (debido al polimorfismo).

Obsérvese las llamadas *madre2.llamame()* de las líneas 36 en adelante:

- En el primero se invoca al método *llamame()* de la clase Madre porque 'madre2' hace referencia a un objeto de la clase Madre.
- En el segundo se invoca al método *llamame()* de la clase Hija1 porque ahora 'madre2' hace referencia a un objeto de la clase Hija1.
- En el tercero se invoca al método *llamame()* de la clase Hija2 porque ahora 'madre2' hace referencia a un objeto de la clase Hija2.

```
class Madre {  
    void llamame(){  
        System.out.println("Estoy en la clase Madre");  
    }  
}  
  
class Hija1 extends Madre {  
    void llamame(){  
        System.out.println("Estoy en la subclase Hija1");  
    }  
}  
  
class Hija2 extends Madre {  
    void llamame(){  
        System.out.println("Estoy en la subclase Hija2");  
    }  
}  
  
class Ejemplo {  
    public static void main(String args[]){  
        // Creamos un objeto de cada clase  
        Madre madre = new Madre();  
        Hija1 h1 = new Hija1();  
        Hija2 h2 = new Hija2();  
  
        // Declaramos otra variable de tipo Madre  
        Madre madre2;  
  
        // Asignamos a madre2 el objeto madre  
        madre2 = madre;  
        madre2.llamame();  
  
        // Asignamos a madre2 el objeto h1 (Hija1)  
        madre2 = h1;  
        madre2.llamame();  
  
        // Asignamos a madre2 el objeto h2 (Hija2)  
        madre2 = h2;  
        madre2.llamame();  
    }  
}
```

Salida:

```
run:  
Estoy en la clase Madre  
Estoy en la subclase Hija1  
Estoy en la subclase Hija2  
BUILD SUCCESSFUL (total time: 0 seconds)
```

4.2 Tipos de polimorfismo

Visto el ejemplo de introducción, vamos a profundizar explicando los mecanismos clave en que se basa el polimorfismo:

- 1) **Sobrecarga** (polimorfismo paramétrico o ad-hoc).
- 2) **Sobreescritura** (polimorfismo de inclusión).
- 3) **Variables polimórficas**

1) Sobrecarga

La sobrecarga de un método (overload) se llama también polimorfismo estático y se produce cuando a métodos con el mismo nombre le variamos la firma de sus parámetros, es decir, si tiene:

- Diferente cantidad de parámetros (lo habitual).
- Misma cantidad pero cambiando uno o varios tipos.
- Misma cantidad y tipo, pero cambiando el orden (poco habitual).

En cualquiera de estos tres casos debemos devolver el mismo tipo. Si no, no lo llamaríamos sobrecarga. El ejemplo más claro de sobrecarga que vimos ya introducido en el tema anterior fue el uso de varios constructores de clase. Veamos ahora otro ejemplo con el método sumar de la clase "Calculadora":

```
class Calculadora {  
    // Método para sumar 2 enteros  
    public int sumar(int a, int b) {  
        return a + b;  
    }  
  
    // Método sobrecargado para sumar 3 enteros  
    public int sumar(int a, int b, int c) {  
        return a + b + c;  
    }  
  
    // Método sobrecargado para sumar 2 números decimales  
    public double sumar(double a, double b) {  
        return a + b;  
    }  
}
```

2) Sobreescritura

La sobreescritura de un método consiste en escribir un método en la subclase que tenga la misma "firma" que el de la superclase y también que devuelva el mismo tipo de dato (o al menos un subtipo de este). En este caso, para la subclase, no se ejecutaría el método padre, se ejecutaría el sobrescrito en ella misma.

Algo que también debemos mencionar es que podemos identificar un método sobrescrito cuando tiene la anotación **@Override**, sin embargo, no es obligatorio ponerlo, pero si es muy recomendable (pues de esta manera el compilador reconoce que este método está sobrescribiendo un método de una superclase de ésta, ayudando a que, si nos equivocamos al construirlo, el compilador nos avise). Adicionalmente si tenemos la anotación inmediatamente podremos saber que se está aplicando el concepto, algo muy útil cuando trabajamos con código de otras personas.

```
// Clase padre o superclase
class Animal {
    public void hacerSonido() {
        System.out.println("Sonido genérico de un animal...");
    }
}

// Subclase que hereda de Animal
class Perro extends Animal {

    @Override
    public void hacerSonido() {
        System.out.println("Guau, guau!");
    }
}

// Otra subclase que hereda de Animal
class Gato extends Animal {

    @Override
    public void hacerSonido() {
        System.out.println("Miau, miau!");
    }
}

public class Main {
    public static void main(String[] args) {
        Animal a1 = new Perro();
        Animal a2 = new Gato();

        // Llamadas a los métodos polimórficos sobreescritos "hacerSonido"
        a1.hacerSonido(); // Mostrará "Guau guau"
        a2.hacerSonido(); // Mostrará "Miau miau"
    }
}
```

Como vimos al principio del apartado, el polimorfismo en Java se manifiesta en tiempo de ejecución. Esto significa que la decisión sobre qué método o clase utilizar se toma durante la ejecución del programa, proporcionando una flexibilidad dinámica y poderosa.

3) Variables polimórficas

Basándonos en la jerarquía de herencia anterior, veamos cómo una variable de un determinado tipo puede referenciar objetos de este tipo, pero también objetos de clases hijas de ese tipo. Estas variables polimórficas pueden ir cambiando de un tipo a otro en tiempo de ejecución (eso sí, no a cualquier tipo, solo dentro de la jerarquía de herencia):

```
Animal a1 = new Animal();
Animal a2 = new Perro();
a1 = new Gato();
```

5. CLASES ABSTRACTAS

Una clase abstracta es **una clase que declara la existencia de algunos métodos pero no su implementación** (es decir, contiene la cabecera del método pero no su código). Los métodos sin implementar son métodos abstractos. Una clase abstracta puede contener tanto métodos abstractos (sin implementar) como no abstractos (implementados). Pero **al menos uno debe ser abstracto**. Para declarar una clase o método como abstracto se utiliza el modificador **abstract**.

⚡ Una clase abstracta **no se puede instanciar (no podemos crear objetos)**, pero **sí heredar**. Las subclases tendrán que implementar obligatoriamente el código de los métodos abstractos (a no ser que también se declaren como abstractas).

Las clases abstractas son útiles cuando necesitamos definir una forma generalizada de clase que será compartida por las subclases, dejando parte del código en la clase abstracta (métodos “normales”) y delegando otra parte en las subclases (métodos abstractos).

⚡ No pueden declararse constructores o métodos estáticos abstractos.

La finalidad principal de una clase abstracta es crear clases heredadas a partir de ella. Por ello, en la práctica, es obligatorio aplicar herencia (si no, la clase abstracta no sirve para nada). El caso contrario es una clase *final*, que no puede heredarse como ya hemos visto. Por lo tanto una clase no puede ser *abstract* y *final* al mismo tiempo.

Por ejemplo, esta clase abstracta Principal tienes dos métodos: uno concreto y otro abstracto.

```
public abstract class Principal {  
    // Método concreto con implementación  
    public void metodoConcreto() {  
        ...  
    }  
  
    // Método abstracto sin implementación  
    public abstract void metodoAbstracto();  
}
```

Esta subclase hereda de Principal ambos métodos, pero está obligada a implementar el código del método abstracto:

```
class Secundaria extends Principal {  
    // Implementación concreta  
    @Override  
    public void metodoAbstracto() {  
        ...  
    }  
}
```


6. INTERFACES

Una interfaz es una **declaración de atributos y métodos sin implementación** (sin definir el código de los métodos). Se utilizan para definir el conjunto mínimo de atributos y métodos de las clases que implementen dicha interfaz. En cierto modo, es parecido a una clase abstracta con todos sus miembros abstractos. Si una clase es una plantilla para crear objetos, **una interfaz es una plantilla para crear clases**.



Un interfaz es una declaración de atributos y métodos sin implementación.

Mediante la construcción de una interfaz, el programador pretende especificar qué caracteriza a una colección de objetos e, igualmente, especificar qué comportamiento deben reunir los objetos que quieran entrar dentro de sea categoría o colección. En una interfaz también se pueden declarar constantes que definen el comportamiento que deben soportar los objetos que quieran implementar esa interfaz. La sintaxis típica de una interfaz es la siguiente:

```
public interface Nombre {  
    // Declaración de atributos y métodos (sin definir código)  
}
```

Si una interfaz define un tipo pero ese tipo no provee de ningún método, podemos preguntarnos: ¿para qué sirven entonces las interfaces en Java? La implementación (herencia) de una interfaz no podemos decir que evite la duplicidad de código o que favorezca la reutilización de código puesto que realmente no proveen código. En cambio sí podemos decir que reúne las otras dos ventajas de la herencia: favorecer el mantenimiento y la extensión de las aplicaciones. ¿Por qué? Porque **al definir interfaces permitimos la existencia de variables polimórficas y la invocación polimórfica de métodos**.

Un aspecto fundamental de las interfaces en Java es **separar la especificación de una clase (qué hace) de la implementación (cómo lo hace)**. Esto se ha comprobado que da lugar a programas más robustos y con menos errores. Es importante tener en cuenta que:

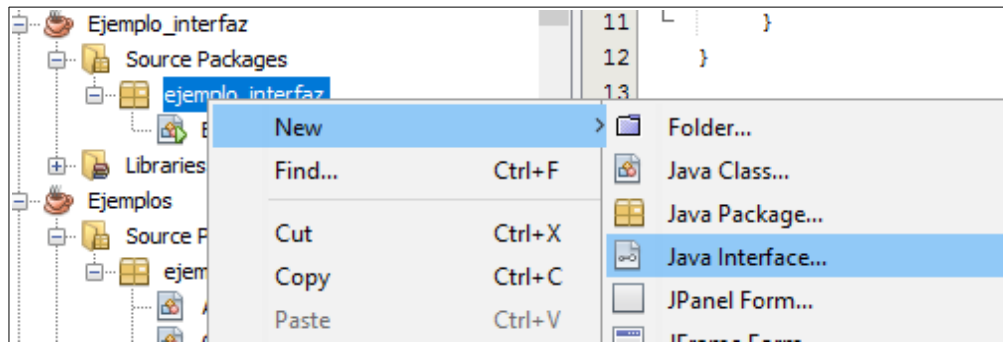
- Una interfaz no se puede instanciar en objetos, solo sirve para implementar clases.
- Una clase puede implementar varias interfaces (separadas por comas).
- Una clase que implementa una interfaz debe de proporcionar implementación para todos y cada uno de los métodos definidos en la interfaz.
- Las clases que implementan una interfaz que tiene definidas constantes pueden usarlas en cualquier parte del código de la clase, simplemente indicando su nombre.

Si por ejemplo la clase *Círculo* implementa la interfaz *Figura* la sintaxis sería:

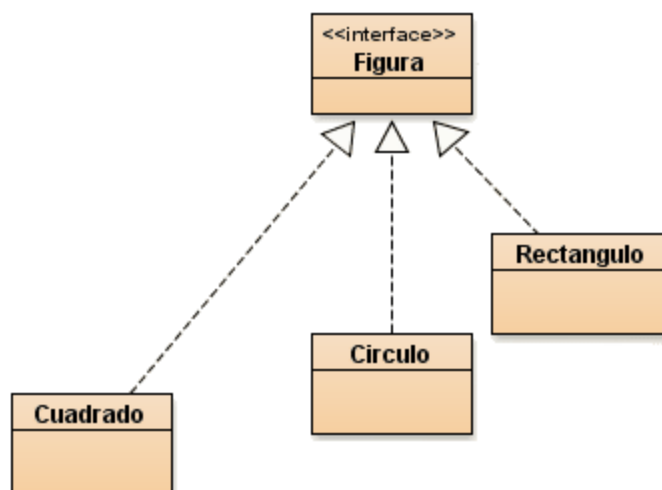
```
public class Círculo implements Figura {  
    ...  
}
```

6.1 Ejemplo básico de interfaces

En este ejemplo vamos a crear una interfaz **Figura** y posteriormente implementarla en varias clases. Para crear una interfaz debemos pinchar con el botón derecho sobre el paquete donde la queramos crear y después **NEW > Java Interface**.



Vamos a ver un ejemplo simple de definición y uso de interfaz en Java. Las clases que vamos a usar y sus relaciones se muestran en el siguiente esquema:



Ahora veamos la implementación de dicho esquema:

```
public interface Figura {  
    float PI = 3.1416f; // Por defecto public static final.  
    float area(); // Por defecto abstract public  
}
```

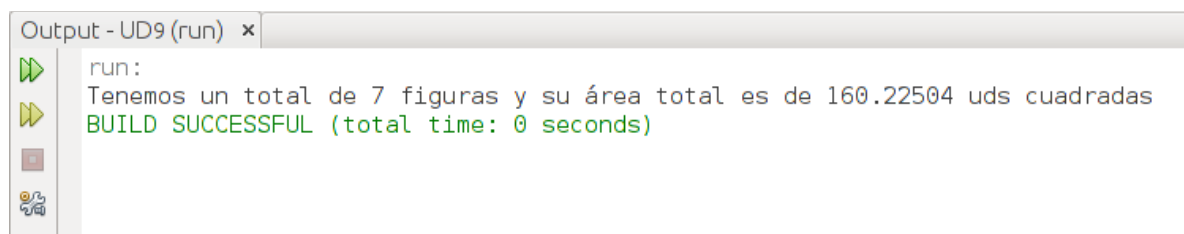
```
12 public class Cuadrado implements Figura {  
13     private float lado;  
14  
15     public Cuadrado (float lado) {  
16         this.lado = lado;  
17     }  
18  
19     public float area() {  
20         return lado*lado;  
21     }  
22 }  
23
```

```
12 public class Rectangulo implements Figura {  
13     private float lado;  
14     private float altura;  
15  
16     public Rectangulo (float lado, float altura) {  
17         this.lado = lado;  
18         this.altura = altura;  
19     }  
20  
21     public float area() {  
22         return lado*altura;  
23     }  
24 }
```

```
12 public class Circulo implements Figura {  
13     private float diametro;  
14  
15     public Circulo (float diametro) {  
16         this.diametro = diametro;  
17     }  
18  
19     public float area() {  
20         return (PI*diametro*diametro/4f);  
21     }  
22 }  
23
```

```
21 public static void main(String[] args) {
22     // TODO code application logic here
23     Figura cuad1 = new Cuadrado (3.5f);
24     Figura cuad2 = new Cuadrado (2.2f);
25     Figura cuad3 = new Cuadrado (8.9f);
26
27     Figura circ1 = new Circulo (3.5f);
28     Figura circ2 = new Circulo (4f);
29
30     Figura rect1 = new Rectangulo (2.25f, 2.55f);
31     Figura rect2 = new Rectangulo (12f, 3f);
32
33     ArrayList serieDeFiguras = new ArrayList();
34
35     serieDeFiguras.add (cuad1);
36     serieDeFiguras.add (cuad2);
37     serieDeFiguras.add (cuad3);
38
39     serieDeFiguras.add (circ1);
40     serieDeFiguras.add (circ2);
41     serieDeFiguras.add (rect1);
42     serieDeFiguras.add (rect2);
43
44     float areaTotal = 0;
45     Iterator it = serieDeFiguras.iterator(); //creamos un iterador
46
47     while (it.hasNext()){
48         Figura tmp = (Figura)it.next();
49         areaTotal = areaTotal + tmp.area();
50     }
51
52     System.out.println ("Tenemos un total de " + serieDeFiguras.size() + " figuras y su área total es de " +
53         areaTotal + " uds cuadradas");
54 }
```

El resultado de ejecución podría ser algo así:



```
Output - UD9 (run) x
run:
Tenemos un total de 7 figuras y su área total es de 160.22504 uds cuadradas
BUILD SUCCESSFUL (total time: 0 seconds)
```

En este ejemplo la **interface “Figura”** define un **tipo de dato**. Por ello podemos crear un ArrayList de figuras donde insertamos cuadrados, círculos, rectángulos, etc. (polimorfismo). Esto nos permite darle un tratamiento común a todas las figuras: Mediante un bucle while recorremos la lista de figuras y llamamos al método `area()` que será distinto para cada clase de figura.

6.2 Ejemplo: interfaz estándar para ordenar objetos complejos

Una de las funciones más útiles de las interfaces para los programadores noveles es en el uso de funcionalidad predefinida por el lenguaje. Cuando vimos la ordenación de ArrayList explicamos que “sort” sólo se podía usar con datos “simples” pero no con las clases creadas por nosotros (por ejemplo, la clase `Persona`). Ahora vamos a ver cómo realizar la ordenación de objetos complejos aplicando criterios específicos soportados por interfaces específicas de Java.

Supongamos que estamos gestionando una lista de objetos complejos, “Persona”, con sus nombres y edades:

```
class Persona {
    private String nombre;
    private int edad;

    public Persona(String nombre, int edad){
        this.nombre = nombre;
        this.edad = edad;
    }
    //etcétera
}

public static void main(String[] args){
    Persona[] personas = new Persona[50];
    personas[0] = new Persona("Ana", 30);
    personas[1] = new Persona("Paco", 40);
    //etcétera
}
```

¿Qué pasa si queremos **ordenar** ese vector por la **edad** de las personas, en orden descendente? Podemos definir un método para ordenar manualmente el vector, aplicando algunos de los algoritmos más conocidos (como el algoritmo de la burbuja, que se verá a final de curso); sin embargo, hay una forma más rápida (en términos de eficiencia) de obtener este resultado. Hay un par de interfaces disponibles en el núcleo de Java que nos permiten clasificar cualquier tipo de objeto. Estas interfaces son **Comparator** y **Comparable**. Ambas tienen un método para ser implementado pero, por su simplicidad, sólo nos centraremos en **Comparable**.

Con esa interfaz debemos implementar un método llamado `compareTo`, que recibe un solo objeto como parámetro y lo compara con el objeto actual (`this`). También devuelve un número entero que indica cuál irá primero en el vector: el objeto actual (número negativo), el objeto recibido como parámetro (número positivo), o cero si son iguales. Como estamos trabajando con `this`, usaremos esta interfaz aplicada a la clase cuyos objetos necesitan ser ordenados.

Veamos cómo usarla con nuestra clase “Persona”. En primer lugar, debemos implementar la interfaz elegida para nuestra clase con el tipo de dato, y también hay que implementar mediante sobreescritura el método “`compareTo`”. En él, devolvemos un número dependiendo de qué edad es mayor: recuerda, debemos clasificar a las personas por edad en orden descendente, por lo que debemos devolver un número negativo si este objeto es más mayor que el parámetro:

```
class Persona implements Comparable<Persona>{
    private String nombre;
    private int edad;
```

```
public Persona(String nombre, int edad){
    this.nombre = nombre;
    this.edad = edad;
}
// Setters y getters

@Override
public int compareTo(Persona p){
    if (this.getEdad() > p.getEdad())
        return -1;
    else if (p.getEdad() > this.getEdad())
        return 1;
    else
        return 0;
}
```

Luego, en nuestro programa principal, solo necesitamos llamar al método `Arrays.sort` o `Collections.sort`, y luego el array/ArrayList se ordenará automáticamente y sabrá que debe hacerlo usando el criterio establecido por el `compareTo` (en el ejemplo anterior es por edad), si cambiamos ese criterio estaríamos cambiando el resultado que nos ofrece “sort”:

```
import java.util.Arrays; // y ArrayList

public static void main(String[] args){
    Persona[] personas = new Persona[50];
    Arrays.sort(personas); // ordenado por edad (descendente)

    ArrayList<Persona> empleados = new ArrayList<>();
    Collections.sort(empleados); // ordenado por edad (descendente)
}
```

 Sugerencia: para asegurarte de haberlo entendido te proponemos algunas modificaciones diferentes:

- a) Que la ordenación sea ascendente por edad en vez de descendente
- b) Ordena por el nombre en vez de por la edad, en orden descendente y luego ascendente.